

深圳大学实验报告

课程名称： 数据挖掘

实验项目名称： 房价预测

学院： 计算机与软件学院

专业： 计算机科学与技术（创新班）

指导教师： 王祎乐

报告人： 李文俊 学号： 2023150001 班级： 高性能

实验时间： 2026年5月22日

实验报告提交时间： 2026年5月22日

教务处制

实验目的与要求：

- 1、了解 Kaggle 平台及其房价预测竞赛任务流程。
- 2、熟悉特征工程的基本步骤，掌握异常值检测、目标变换、缺失值处理、特征组合、类别编码与特征降维等数据分析与数据处理手段。
- 3、能熟练使用经典的回归模型解决连续值预测问题，并在此基础上使用 Bagging、Boosting 与 Stacking 等集成学习方法进一步提升预测性能

试验环境：

- 1、操作系统：macOS
- 2、Python 环境：实验目录本地虚拟环境 `./venv` (venv)
- 3、Python 版本：3.14.3
- 4、核心库版本：numpy 2.4.6、pandas 3.0.3、matplotlib 3.10.9、seaborn 0.13.2、scipy 1.17.1、scikit-learn 1.8.0、xgboost 3.2.0、lightgbm 4.6.0

其中：

pandas 用于 CSV 数据读取、训练集与测试集合并以及最终 submission.csv 导出；

numpy 提供数值计算与 `log/expm1` 等数值变换；

scipy.stats 用于绘制 SalePrice 的正态拟合与 QQ 概率图、并通过 skew 计算偏度，scipy.special.boxcox1p 用于对高偏度特征做 Box-Cox 修正；

scikit-learn 提供 LabelEncoder/get_dummies 类别编码、RobustScaler 鲁棒标准化、KFold/cross_val_score 五折交叉验证、LinearRegression/Ridge/Lasso/ElasticNet/KernelRidge 等回归模型，以及 BaggingRegressor/RandomForestRegressor/AdaBoostRegressor/GradientBoostingRegressor 等集成模型，并通过 BaseEstimator/RegressorMixin/TransformerMixin/clone 自定义 Averaging 与 Stacking 元模型；

xgboost 与 lightgbm 分别提供 XGBRegressor 与 LGBMRegressor 两个高性能梯度提升库；

matplotlib 与 seaborn 负责绘制异常值散点图、SalePrice 分布曲线与 QQ 图、缺失值占比柱状图、特征相关性热力图与模型 RMSE 对比柱状图。

实验内容及过程：

1. 实验任务与数据说明

本实验基于 Kaggle 房价预测 (House Prices: Advanced Regression Techniques) 竞赛数据，任务是根据爱荷华州埃姆斯地区住宅的 79 个属性特征，预测每间房屋的销售价格 SalePrice。

1.1 数据文件说明

实验使用官方数据集，解压后包含四个文件，均存放于 ./dataset/ 目录下。

- (1) train.csv: 训练集，1460 条样本，81 列字段，其中 80 个为房屋特征，最后一列 SalePrice 为售价标签
- (2) test.csv: 测试集，1459 条样本，80 列字段，仅包含房屋特征，不含 SalePrice，是需要预测并提交的样本
- (3) data_description.txt: 特征字典，描述了 80 个字段的含义及其可能取值，如 MSZoning、PoolQC、Neighborhood 等
- (4) sample_submission.csv: 提交格式示例，每行包含 Id 与 SalePrice 两列

1.2 评价指标

本任务为典型的回归任务，官方采用对预测价格与真实价格分别取对数后的均方根误差 **RMSE** 作为评价指标，本实验在**交叉验证阶段**直接以 $\log_{10}(\text{SalePrice})$ 为目标，使用 sklearn 的 `neg_mean_squared_error` 评分，再开根号得到 5 折交叉验证 RMSE，RMSE 越小说明模型在 $\log$ 空间下的预测越准确。

2. 数据加载与异常值检测

2.1 数据加载

使用 `pandas.read_csv` 加载 train.csv 与 test.csv，保存 Id 列后将其从特征中剔除，得到 train shape=(1460, 81)、test shape=(1459, 80)。

```
train = pd.read_csv(os.path.join(DATA_DIR, 'train.csv'))
test = pd.read_csv(os.path.join(DATA_DIR, 'test.csv'))
train_id = train['Id']; test_id = test['Id']
train.drop('Id', axis=1, inplace=True); test.drop('Id', axis=1, inplace=True)
```

图 1 数据集读取与 Id 列分离代码

2.2 异常值检测

首先绘制 GrLivArea 与 SalePrice 的散点图，在右下角可见两个明显偏离的样本：居住面积大于 4000 平方英尺、但售价却低于 30 万美元，这与一般房价市场规律不符。

按照实验指导书的建议，将这两个异常样本从训练集中移除，以避免它们对回归模型造成过拟合干扰。

```
train = train.drop(train[(train['GrLivArea']>4000) & (train['SalePrice']<300000)].index)
```

图 2 GrLivArea 与 SalePrice 异常值剔除代码

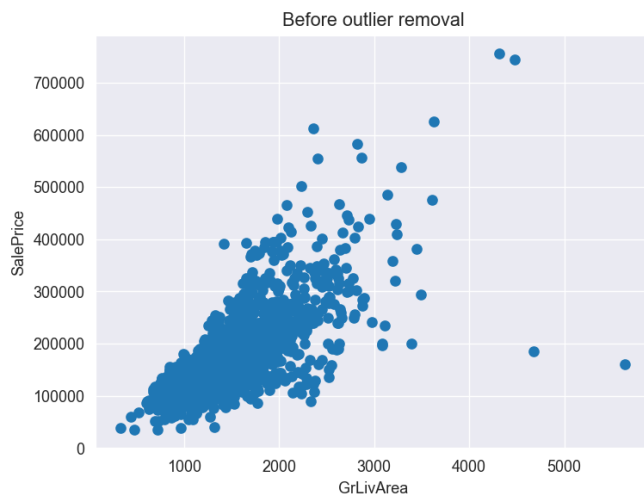


图 3 剔除前 GrLivArea-SalePrice 散点图

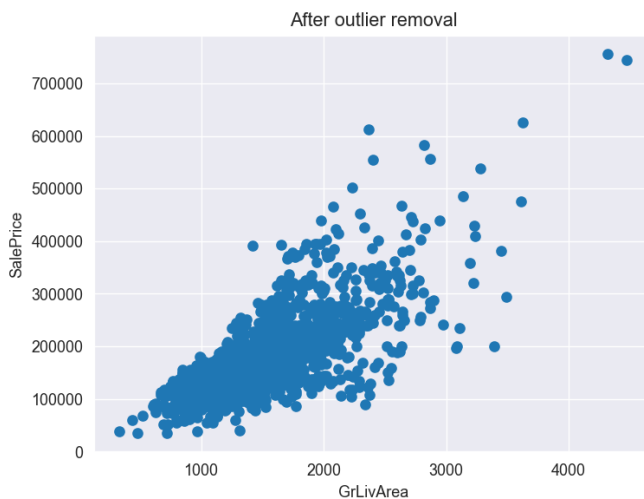


图 4 剔除后 GrLivArea-SalePrice 散点图

2.3 目标变量的正态化变换

对 SalePrice 拟合正态分布并绘制 QQ 概率图，可观察到原始 SalePrice 整体右偏，与正态分布偏差较大。

由于线性模型与高斯核回归对目标变量近似正态分布更为敏感，因此使用 `np.log1p` 对其做 $\log(1+x)$ 变换，变换后均值 $\mu \approx 12.02$ 、标准差 $\sigma \approx 0.40$ ，分布与正态曲线高度吻合，QQ 图上的样本点也近似落在 $y=x$ 上。

```
train['SalePrice'] = np.log1p(train['SalePrice'])
```

图 5 log1p 目标变换代码

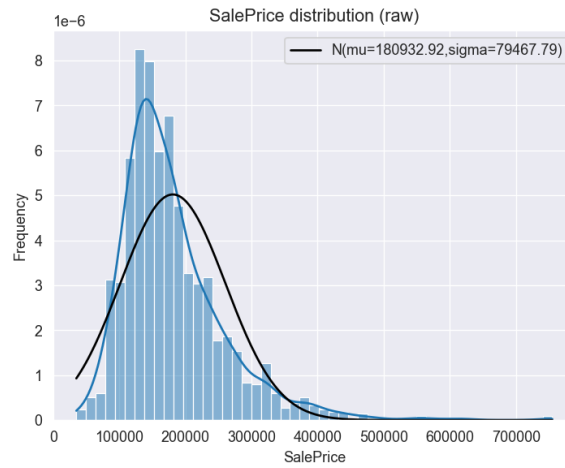


图 6 原始 SalePrice 分布

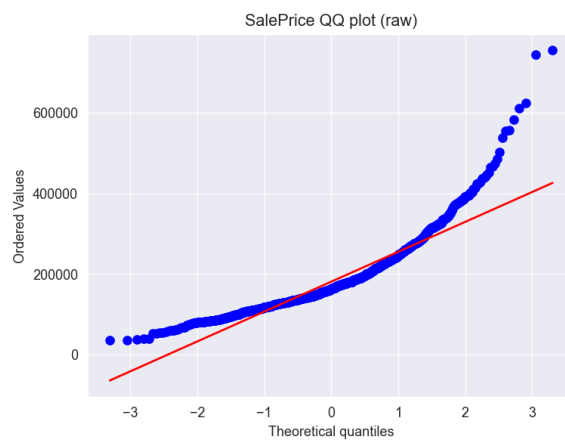


图 7 原始 SalePrice QQ 概率图

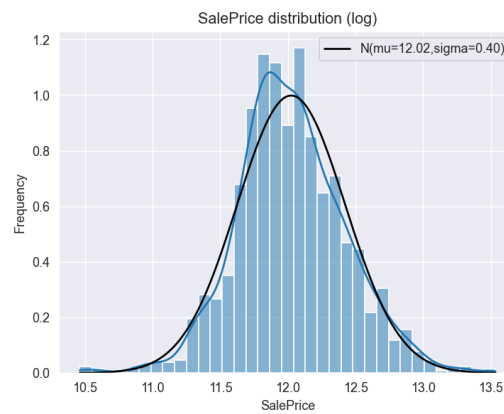


图 8 log1p 变换后 SalePrice 分布

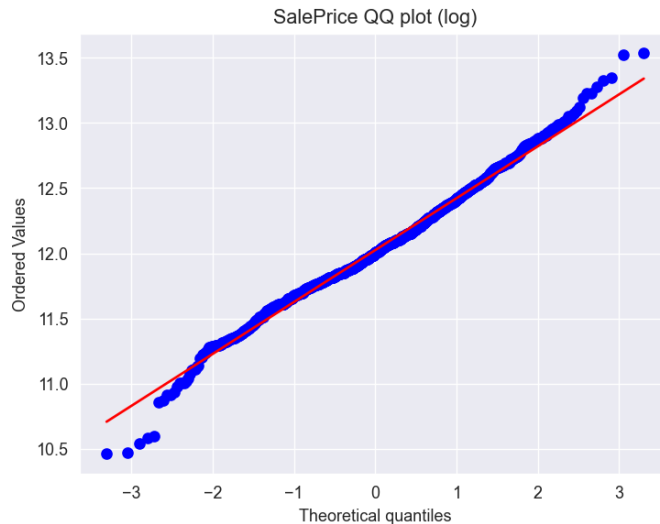


图 9 log1p 变换后 SalePrice QQ 概率图

3. 特征工程

3.1 训练集与测试集合并

由于训练集和测试集中均存在缺失值与待编码字段，将二者纵向拼接为 `all_data` 一并处理，可保证两侧的填充值、编码字典与降维投影空间完全一致。合并后 `all_data` 共 2917 行、79 列。

```
ntrain = train.shape[0]
y_train = train['SalePrice'].values
all_data = pd.concat((train,
test)).reset_index(drop=True)
all_data.drop(['SalePrice'], axis=1, inplace=True)
```

图 10 合并训练集与测试集代码

3.2 缺失值统计与填充

首先统计各特征缺失率，并绘制缺失率柱状图：

- (1) 可以看到 `PoolQC`、`MiscFeature`、`Alley`、`Fence` 这类描述“房屋是否具备某项设施”的字段缺失率高达 **80%** 以上，含义为该房屋没有该设施，因此用 `None` 填充；
- (2) `GarageArea`、`TotalBsmtSF`、`MasVnrArea` 等数值字段缺失，含义为对应**设施面积为 0**，因此用 `0` 填充；
- (3) `LotFrontage` 表示街道临街面长度，因相邻房屋常具有相似的临街面，故按 `Neighborhood` 分组取中位数填充；
- (4) `MSZoning`、`Electrical`、`KitchenQual`、`Exterior1st/2nd`、`SaleType` 等仅个别样本缺失，用**众数**填充；
- (5) `Functional` 缺失则按数据描述默认填入 `Typ`（典型）；
- (6) `Utilities` 在训练集中 2916 个样本均为 `AllPub`，对预测无信息量，直接删除该列。

```

for c in ['PoolQC', 'MiscFeature', 'Alley', 'Fence', 'FireplaceQu', ...]:
    all_data[c] = all_data[c].fillna('None')
for c in ['GarageYrBlt', 'GarageArea', 'GarageCars', 'BsmtFinSF1', 'BsmtFinSF2',
          'BsmtUnfSF', 'TotalBsmtSF', 'BsmtFullBath', 'BsmtHalfBath', 'MasVnrArea']:
    all_data[c] = all_data[c].fillna(0)
all_data['LotFrontage'] = all_data.groupby('Neighborhood')['LotFrontage']\
    .transform(lambda x: x.fillna(x.median()))
all_data = all_data.drop(['Utilities'], axis=1)
all_data['Functional'] = all_data['Functional'].fillna('Typ')

```

图 11 缺失值填充代码

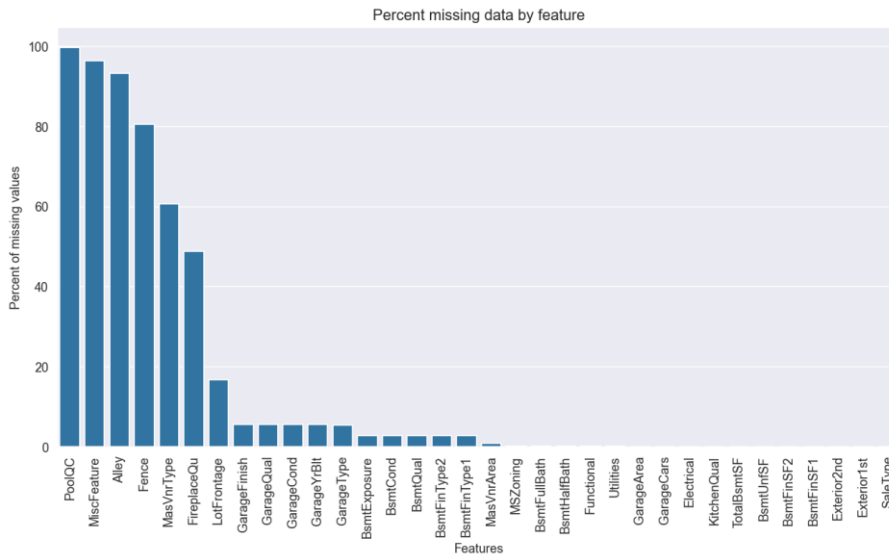


图 12 填充前各特征缺失率柱状图

缺失值填充完成后，再次扫描整张表，all_data 已不存在任何缺失字段，可以安全进入后续数值变换阶段。

3.3 高偏度特征的 Box-Cox 变换

对所有数值型特征 (select_dtypes(include=[np.number])) 计算偏度 skew，并保留 |skew| > 0.75 的特征作为待变换集合，本实验中得到 **21 个高偏度特征**，其中 MiscVal、PoolArea、LotArea、LowQualFinSF、3SsnPorch 偏度均超过 11，分布严重右偏。对这些特征采用 boxcox1p(x, lam) 进行 **Box-Cox** 一阶变换，设 $\lambda=0.15$ ，使其分布更接近正态，降低长尾值对回归模型的扰动。

```

numeric_feats = all_data.select_dtypes(include=[np.number]).columns
skewed = all_data[numeric_feats].apply(lambda x: skew(x.dropna()))\
    .sort_values(ascending=False)
skewness = skewed[abs(skewed) > 0.75]
for feat in skewness.index:
    all_data[feat] = boxcox1p(all_data[feat], 0.15)

```

图 13 高偏度特征 Box-Cox 变换代码

3.4 特征组合

通过 data_description.txt 可知 TotalBsmtSF (地下室面积)、1stFlrSF (一楼面积)、2ndFlrSF (二楼面积) 共同度量房屋的总居住面积，三者之和与房价高度相关，但单独使用时存在冗余。

因此构造一个新的衍生特征 **TotalSF**，等于上述三者之和，作为模型可直接利用的“**总面积**”信号。

```

all_data['TotalSF'] = all_data['TotalBsmtSF'] + all_data['1stFlrSF'] + all_data['2ndFlrSF']

```

图 14 TotalSF 衍生特征构造代码

3.5 类别特征编码

数据集中部分数值型字段（如 MSSubClass、OverallCond、YrSold、MoSold）实际语义为类别，若按数值参与模型训练会引入伪线性关系，因此先用 `astype(str)/apply(str)` 将其转为字符串；

随后对 26 个有序类别特征（如 FireplaceQu、BsmtQual、ExterQual、HeatingQC 等）使用 `LabelEncoder` 做有序编码；

最后对剩余的无序类别特征使用 `pd.get_dummies` 进行独热编码。完成后特征矩阵从 79 列扩展到 220 列，所有列均为数值，可直接喂给回归模型。

```
all_data['MSSubClass'] = all_data['MSSubClass'].apply(str)
all_data['OverallCond'] = all_data['OverallCond'].astype(str)
all_data['YrSold'] = all_data['YrSold'].astype(str)
all_data['MoSold'] = all_data['MoSold'].astype(str)
for c in cols:
    lbl = LabelEncoder().fit(list(all_data[c].values))
    all_data[c] = lbl.transform(list(all_data[c].values))
all_data = pd.get_dummies(all_data)
```

图 15 LabelEncoder 与 get_dummies 联合编码代码

3.6 PCA 特征降维

独热后特征维度上升到 220，**存在大量低方差列与高度相关字段**（如 GarageCars 与 GarageArea）。

调用 `sklearn.decomposition.PCA(n_components=100)` 对 `all_data` 做主成分分解，保留 100 个主成分，可在保持几乎全部方差信息的同时去除特征间的线性相关性。

PCA 在此处主要作为相关性诊断与可选输入，最终模型仍采用 220 维原始特征进行训练。

```
pca = PCA(n_components=100, random_state=42)
all_data_pca = pca.fit_transform(all_data)
print(all_data_pca.shape, pca.explained_variance_ratio_.sum())
```

图 16 PCA 降维代码

3.7 相关性分析

为了直观观察特征之间以及特征与目标 `SalePrice` 之间的关联强度，对编码后的 `train` 部分计算 Pearson 相关系数矩阵，并使用 `seaborn.heatmap` 绘制热力图。

热力图中可以看到 `GarageCars` 与 `GarageArea`、`TotalBsmtSF` 与 `1stFlrSF`、`OverallQual` 与 `SalePrice` 等都呈现出明显的正相关，验证了 `TotalSF` 衍生特征的合理性。

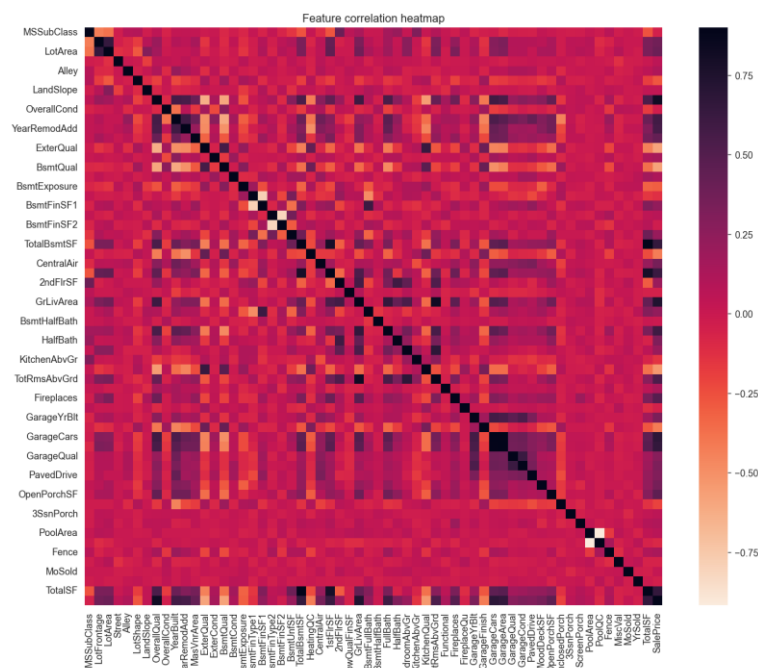


图 17 特征相关性热力图

4. 模型训练与评估

4.1 五折交叉验证

为避免单次划分带来的方差，采用 `sklearn.model_selection.KFold(n_splits=5, shuffle=True, random_state=42)` 构造五折交叉验证，使用 `neg_mean_squared_error` 作为评分函数，对结果取负数再开根号得到每折的 RMSE。

`rmsle_cv` 函数返回 5 个折的 RMSE 数组，便于同时输出均值与标准差以衡量稳定性。

```
def rmsle_cv(model, X, y, n_folds=5):
    kf = KFold(n_splits=n_folds, shuffle=True, random_state=42).get_n_splits(X)
    rmse = np.sqrt(-cross_val_score(model, X, y, scoring='neg_mean_squared_error', cv=kf))
    return rmse
```

图 18 五折交叉验证评分函数代码

4.2 基础回归模型

依次构造五个经典回归模型，对易受量纲影响的线性模型在前面接入 `RobustScaler`，用四分位距而非标准差做标准化，对离群点更稳健。具体设置如下：

- (1) **LinearRegression**: 原始最小二乘回归，作为基线。
- (2) **Ridge(alpha=0.9)**: L2 正则化，缓解多重共线性。
- (3) **Lasso(alpha=0.0005)**: L1 正则化，对稀疏特征友好。
- (4) **ElasticNet(alpha=0.0005, l1_ratio=0.9)**: L1+L2 混合正则化。
- (5) **KernelRidge(alpha=0.6, kernel='polynomial', degree=2, coef0=2.5)**: 多项式核岭回归。

```
linear = make_pipeline(RobustScaler(), LinearRegression())
ridge = make_pipeline(RobustScaler(), Ridge(alpha=0.9, random_state=1))
lasso = make_pipeline(RobustScaler(), Lasso(alpha=0.0005, random_state=1))
ENet = make_pipeline(RobustScaler(), ElasticNet(alpha=0.0005, l1_ratio=0.9,
random_state=1))
KRR = KernelRidge(alpha=0.6, kernel='polynomial', degree=2, coef0=2.5)
```

图 19 五个基础回归模型构造代码

五折交叉验证 RMSE 结果如下：

- (1) **LinearRegression: 0.1208 (± 0.0094)**
- (2) **Ridge: 0.1160 (± 0.0087)**
- (3) **Lasso: 0.1107 (± 0.0068)**
- (4) **ElasticNet: 0.1106 (± 0.0069)**
- (5) **KernelRidge: 0.1825 (± 0.0149)**

4.3 Bagging 集成

Bagging 通过对样本进行有放回采样训练多个基模型并取平均，可显著降低模型方差。

实验构造两个 Bagging 类模型：以 LinearRegression 为基学习器的 BaggingRegressor，以及以决策树为基学习器、共 100 棵的 RandomForestRegressor。

```
Bagging = BaggingRegressor(estimator=LinearRegression(), n_estimators=10,
random_state=1)
RandomForest = RandomForestRegressor(n_estimators=100, random_state=42)
```

图 20 Bagging 与 RandomForest 构造代码

五折交叉验证 RMSE 结果如下：

- (1) **Bagging(LR): 0.1207 (± 0.0091)**
- (2) **RandomForest: 0.1386 (± 0.0055)**

4.4 Boosting 集成

Boosting 通过串行训练基学习器，让后续模型重点拟合前者的残差，能在偏差与方差之间取得较好折衷。

本实验构造四个 Boosting 模型：

- (1) AdaBoostRegressor (以 LinearRegression 为基学习器)
- (2) GradientBoostingRegressor (GBDT, 3000 棵树, huber 损失)
- (3) XGBRegressor (2200 棵树, 含 colsample_bytree、reg_alpha、reg_lambda 等正则项)
- (4) LGBMRegressor (720 棵树, 使用 leaf-wise 生长, feature_fraction 与 bagging_fraction 控制随机性)

```

AdaBoost = AdaBoostRegressor(estimator=LinearRegression(), n_estimators=50,
                             learning_rate=1.0, loss='linear', random_state=42)
GBoost = GradientBoostingRegressor(n_estimators=3000, learning_rate=0.05,
                                   max_depth=4, max_features='sqrt',
                                   min_samples_leaf=15, min_samples_split=10,
                                   loss='huber', random_state=5)
model_xgb = xgb.XGBRegressor(colsample_bytree=0.4603, gamma=0.0468,
                             learning_rate=0.05, max_depth=3,
                             min_child_weight=1.7817, n_estimators=2200,
                             reg_alpha=0.4640, reg_lambda=0.8571,
                             subsample=0.5213, random_state=7, n_jobs=-1, verbosity=0)
model_lgb = lgb.LGBMRegressor(objective='regression', num_leaves=5,
                              learning_rate=0.05, n_estimators=720, max_bin=55,
                              bagging_fraction=0.8, bagging_freq=5,
                              feature_fraction=0.2319, min_data_in_leaf=6,
                              min_sum_hessian_in_leaf=11, verbose=-1)

```

图 21 AdaBoost / GBDT / XGBoost / LightGBM 构造代码

五折交叉验证 RMSE 结果如下：

- (1) **AdaBoost(LR): 0.1644 (± 0.0083)**
- (2) **GradientBoosting: 0.1163 (± 0.0080)**
- (3) **XGBoost: 0.1180 (± 0.0066)**
- (4) **LightGBM: 0.1165 (± 0.0063)**

4.5 Stacking 融合

为进一步压低误差，引入两层模型融合策略：

- (1) **首先实现 AveragingModels**：对若干已 clone 的基模型分别 fit，预测时取算术平均，使用 (ENet, GBoost, KRR, Lasso) 作为 Averaging 基模型
- (2) **再实现 StackingAveragedModels**：使用五折交叉验证生成基模型的 out-of-fold 预测，并以这些 OOF 预测作为新特征训练元模型 Lasso；使用 (ENet, GBoost, KRR) 作为 Stacking 基模型，并以 Lasso 作为元模型
- (3) **最后预测时**对每个基模型的克隆体取平均后送入元模型。

```

class AveragingModels(BaseEstimator, RegressorMixin, TransformerMixin):
    def __init__(self, models): self.models = models
    def fit(self, X, y):
        self.models_ = [clone(m) for m in self.models]
        for m in self.models_: m.fit(X, y)
        return self
    def predict(self, X):
        return np.mean(np.column_stack([m.predict(X) for m in self.models_]), axis=1)

class StackingAveragedModels(BaseEstimator, RegressorMixin, TransformerMixin):
    def __init__(self, base_models, meta_model, n_folds=5): ...
    def fit(self, X, y):
        kfold = KFold(n_splits=self.n_folds, shuffle=True, random_state=156)
        oof = np.zeros((X.shape[0], len(self.base_models)))
        for i, m in enumerate(self.base_models):
            for tr, ho in kfold.split(X, y):
                inst = clone(m); inst.fit(X[tr], y[tr])
                self.base_models_[i].append(inst)
                oof[ho, i] = inst.predict(X[ho])
        self.meta_model_.fit(oof, y)
        return self

```

图 22 AveragingModels 与 StackingAveragedModels 实现代码

五折交叉验证 RMSE 结果如下：

- (1) **AveragedBaseModels: 0.1137 (± 0.0076)**
- (2) **StackingAveragedModels: 0.1079 (± 0.0073)**

5. 最终加权融合与结果分析

5.1 加权融合方案

将 StackingAveragedModels、XGBoost、LightGBM 三者在全量训练集上各自 fit 一次,再以 0.70/0.15/0.15 的权重对预测的 log SalePrice 进行加权融合,最后用 np.expml 反 log1p 得到真实价格,写入 submission.csv 作为最终提交结果。

三模型在训练集上的 RMSE 与加权融合后的训练 RMSE 如下:

- (1) Stacking 训练 RMSE: 0.0806
- (2) XGBoost 训练 RMSE: 0.0867
- (3) LightGBM 训练 RMSE: 0.0721
- (4) 加权融合训练 RMSE: 0.0781

```
stacked.fit(train_x, y_train)
model_xgb.fit(train_x, y_train)
model_lgb.fit(train_x, y_train)
stacked_pred = np.expml(stacked.predict(test_x))
xgb_pred = np.expml(model_xgb.predict(test_x))
lgb_pred = np.expml(model_lgb.predict(test_x))
ensemble = 0.70 * stacked_pred + 0.15 * xgb_pred + 0.15 * lgb_pred
pd.DataFrame({'Id': test_id, 'SalePrice': ensemble}).to_csv('submission.csv', index=False)
```

图 23 加权融合与 submission.csv 导出代码

5.2 模型对比与结果分析

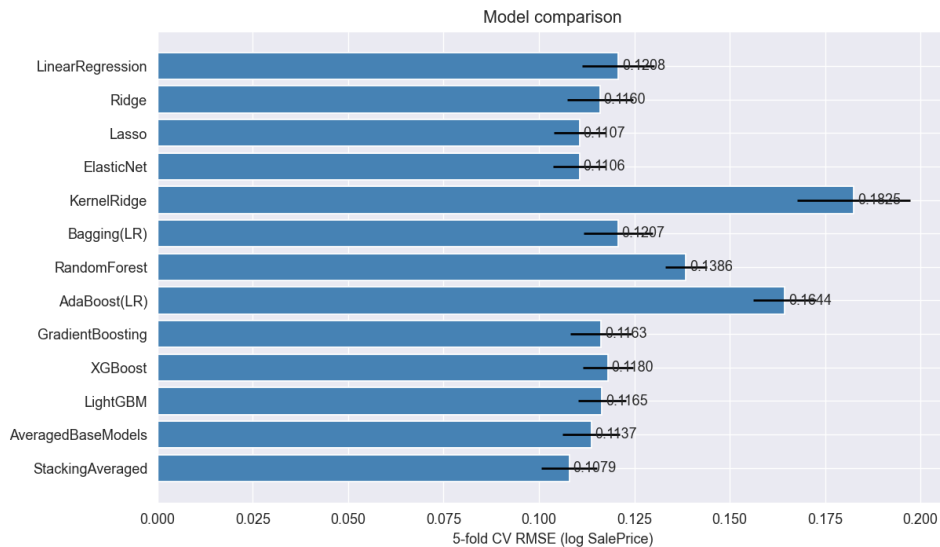


图 24 各模型五折交叉验证 RMSE 对比柱状图

RMSE 越小越好, 结果体现出以下几点:

- (1) 正则化对线性模型有明显帮助: 相比 LinearRegression 的 0.1208, Lasso 与 ElasticNet 分别降到 0.1107 与 0.1106, 说明 220 维独热特征中存在大量噪声列, L1/L1+L2 正则化通过将不重要列的系数压缩到 0 起到了特征筛选作用; Ridge 仅用 L2 正则化下降幅度有限, 验证了稀疏正则的必要性
- (2) 未调参的 KernelRidge 在本实验中表现最差 (0.1825): 原因是多项式核在高维稀疏空间中很容易过拟

合，若调高 α 或换成线性核能更接近其他模型水平

(3) **Bagging(LR) 几乎等同于单个 LinearRegression** (0.1207 vs 0.1208): 说明对低方差的线性模型而言 Bagging 并不能带来额外收益; RandomForest 表现弱于 Lasso 与 ElasticNet (0.1386 vs 0.110), 提示树模型在缺乏深度调参时不如线性模型在该数据集上稳定

(4) **Boosting 系列中, AdaBoost(LR) 退化严重 (0.1644)**: 说明在低偏差线性基模型上 boosting 反而引入了噪声方向; GBDT、XGBoost、LightGBM 三者都达到 0.116 附近, 且 LightGBM 标准差最小, 在精度与稳定性之间取得了较好平衡

(5) **Stacking 是本实验性能最好的策略**: AveragedBaseModels (ENet+GBoost+KRR+Lasso) 降到 0.1137, StackingAveragedModels (以 Lasso 为元模型) 进一步降到 0.1079, 比单个最优基模型再降约 0.003; 最后将 Stacking 与 XGBoost、LightGBM 三者按 0.70/0.15/0.15 加权融合后, 全量训练集 RMSE 为 0.0781, 在保留 Stacking 较低偏差的同时, 借助 XGBoost 与 LightGBM 的不同基学习器结构进一步降低方差。

误差分析:

(1) 本任务样本量较小 (约 1460 条), 独热后维度高达 220, 类别字段中 PoolQC、MiscFeature 等极端稀疏既容易过拟合也容易被正则化清零; 数值字段中即便经过 Box-Cox 变换, 仍存在如 LotArea 这种长尾分布, 对核方法不友好;

(2) 另外训练集与测试集是按时间或抽样切分的, 可能存在分布漂移, 这也是单模型容易在交叉验证与最终 Kaggle 测试集之间出现偏差的原因, 也是采用 Stacking + 加权融合而非单模型预测的根本动机。

实验收获:

一开始我以为这次实验和实验三类似, 读数据、调几个 sklearn 的回归 fit 就能拿到不错的 RMSE。真正动手才发现, House Prices 数据集的难点根本不在模型, 而在前面那段冗长的特征工程: 80 个字段里有的缺失率高达 99%, 有的表面是数字实际是类别, 每一步不仔细都会让 RMSE 出现明显偏差。

缺失值这一步我踩的坑是没分清 “NaN 代表没有” 和 “NaN 代表缺信息”。一开始我对所有列用众数或 0 一刀切, 结果 PoolQC、Alley 被填成最常见等级, 把无泳池的房子伪装成了劣等房。对照 data_description.txt 一列列重新设计填充策略, RMSE 才从 0.13 量级降到 0.11 量级。目标变换同样让我意识到理论重要性: 原始 SalePrice 是长尾分布, $\log_{10}$ 之后 σ 从 8 万降到 0.4, 线性模型才算回到 “线性世界”。

对集成方法的理解也被这次实验重写了。我原以为把 RandomForest、XGBoost 这些 “强模型” 挨个跑一遍取最好的即可, 实测却是 ElasticNet (0.1106) 反而强过 RandomForest (0.1386) —— 因为样本不大、独热后特征稀疏, 树模型拿不到主信号, 正则化反而能直接压零无效列。最终最佳的 0.1079 来自 Stacking(ENet+GBoost+KRR $\rightarrow$ Lasso), 这证明 “单模型选最强、集成选互补” 才是正确顺序。

Stacking 实现我踩了两次坑: 先是用全量训练集预测拼新特征, 导致典型数据泄漏; 改用 out-of-fold 预测后元模型才学到真正的纠错信号。再是 0.70/0.15/0.15 这种权重并不能无脑设置, 本质上是让低方差的 Stacking 做主干, XGBoost 与 LightGBM 提供独立扰动以降低相关误差。这是本次实验我获得的最重要的底层收获。

指导教师批阅意见:

成绩评定:

指导教师签字:

年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。